3665. Twisted Mirror Path Count

Daily Question - October 31, 2026

Problem Statement

Given an m x n binary grid, grid[i][j] = 0 represents an empty cell and grid[i][j] = 1 represents a mirror. A robot starts at (0, 0) and wants to reach (m - 1, n - 1). It can move only right or down.

When the robot attempts to enter a mirror while moving right, it is reflected downward into the cell directly below that mirror. When it attempts to enter a mirror while moving down, it is reflected rightward into the cell directly to the right of that mirror.

If the next cell is also a mirror, reflection continues immediately according to the current direction. The process continues until an empty cell or the destination is reached, or the robot moves outside the grid. Any path that leaves the grid is invalid.

Return the number of unique valid paths from the start to the destination, modulo 1,000,000,007. Different choices that later merge into the same cell are still distinct paths.

Examples

Example 1: grid = [[0,1,0],[0,0,1],[1,0,0]]. Output: 5.

Example 2: grid = [[0,0],[0,0]]. Output: 2. The paths are right then down, and down then right.

Example 3: grid = [[0,1,1],[1,1,0]]. Output: 1. Starting right leads through mirrors (0,1) and (1,1) to the destination (1,2). Starting down leads through mirrors (1,0) and (1,1), then outside the grid.

Constraints

m == grid.length; n == grid[i].length; 2 <= m, n <= 500; grid[i][j] is 0 or 1; grid[0][0] == grid[m - 1][n - 1] == 0.

Solution: Direction-Aware Dynamic Programming

The number of valid continuations at a mirror depends on the direction of approach. Track two states per cell: rightEntry[r][c], for approaching while moving right, and downEntry[r][c], for approaching while moving down. These are conceptual arrival states; a mirror is not a place where the robot can stop and choose a move.

Empty cell: either next move is allowed. Both states equal (rightEntry[r][c + 1] + downEntry[r + 1][c]) modulo MOD.

Mirror cell: a rightward approach forces a downward move, so rightEntry[r][c] = downEntry[r + 1][c]. A downward approach forces a rightward move, so downEntry[r][c] = rightEntry[r][c + 1].

Destination: set both states to 1 because reaching it completes one path. Use an extra zero-filled row and column so out-of-bounds moves contribute zero. Process cells from bottom-right to top-left. The starting cell is empty, so either of its two states gives the total answer; do not add them together.

Complete Java Solution

```
class Solution {
    private static final int MOD = 1_000_000_007;

    public int uniquePaths(int[][] grid) {
        int rows = grid.length;
        int columns = grid[0].length;

        int[][] rightEntry = new int[rows + 1][columns + 1];
        int[][] downEntry = new int[rows + 1][columns + 1];

        for (int row = rows - 1; row >= 0; row--) {
            for (int column = columns - 1; column >= 0; column--) {
                if (row == rows - 1 && column == columns - 1) {
                    rightEntry[row][column] = 1;
                    downEntry[row][column] = 1;
                    continue;
                }

                if (grid[row][column] == 1) {
                    // Rightward approach: turn down.
                    rightEntry[row][column] =
                            downEntry[row + 1][column];

                    // Downward approach: turn right.
                    downEntry[row][column] =
                            rightEntry[row][column + 1];
                } else {
                    int ways = (
                            rightEntry[row][column + 1]
                            + downEntry[row + 1][column]
                    ) % MOD;

                    rightEntry[row][column] = ways;
                    downEntry[row][column] = ways;
                }
            }
        }

        return rightEntry[0][0];
    }
}
```

Correctness

At the destination, each state correctly counts one completed path. Suppose the states below and to the right of a cell are correct. For a mirror, its approach direction determines exactly one forced continuation, so the corresponding recurrence counts precisely the valid paths. For an empty cell, every path starts with either a right move or a down move. These are distinct choices, so adding their continuation counts is correct. Out-of-bounds continuations contribute zero. By reverse induction over the cells, every state is correct, including the returned state at the empty starting cell.

Complexity

Time: O(m * n), since each cell requires constant work. Auxiliary space: O(m * n) for the two DP tables. The input grid is not modified.

Implementation Notes

Consecutive mirrors need no separate loop: the direction-specific states propagate forced reflections automatically. All dependencies move right or down, so no cycles occur. Each stored count is below MOD; adding two such counts fits in a Java int before taking the remainder. Use the exact method signature int

uniquePaths(int[][] grid).